Recovering Navigable Connectivity from IFC: Benchmark, Baselines, and What Is Still Unmeasured

BIM-Graphs Technical Report

BIM-Graphs Project

September 2, 2026

Abstract

Exported IFC states geometry and semantics but not topology: across 25 real Revit exports (257 MB) there are zero IfcRelSpaceBoundary instances, only 51% of storeys contain any IfcSpace, and 12% of elements are assigned to a room. Recovering which rooms exist and which of them actually connect is therefore the problem, and it cannot be scored on real files because no real file states the answer. We build a procedurally generated benchmark whose connectivity is known by construction and independently verified by a second, unrelated mechanism (2,769/2,769 room pairs), and report edge precision/recall against it as a function of how badly the IFC has been damaged. Our method reaches F1 0.996 on clean input and 0.868 under damage, beating the strongest in-repo ablation by +0.085 (131 wins / 0 losses, paired Wilcoxon $p < 10^{-6}$, $n = 150$) and an independently published external tool by +0.495. A graduated damage sweep isolates room recovery as the sole binding constraint. We then report what these numbers do not establish: on the 25 real models we have only necessary conditions, not accuracy, and we show why a partner company's scene-graph delivery — verified accurate to the integer — still cannot substitute for the annotation that would close that gap. This report consolidates every connectivity result measured to date, with the command that reproduces each.

1 The benchmark and why it is synthetic

No real IFC states room-to-room connectivity, so a real corpus offers nothing to score against. main/generate_building.py inverts the problem: the answer is the input. It lays out each floor by recursive rectangular subdivision, then decides, for every geometrically adjacent room pair, whether the pair is actually joined — a door (door_ratio, default 0.6), a door-less opening (open_ratio, default 0.15), or nothing at all. The remainder are solid wall.

Those negatives are the single most important property of the corpus. If every adjacent pair were connected, a trivial "join every neighbour" baseline would score near-perfect and the benchmark could not distinguish any two methods. The current corpus holds 2,049 rooms, 1,708 ground-truth edges (1,293 door, 265 open-passage, 150 vertical) and 1,211 adjacent-but-unconnected negatives across 150 two-storey buildings.

Ground truth is verified, not asserted. The generator and the recovery pipeline share an author, so a shared misconception would produce a perfect score that means nothing. main/verify_gt.py therefore checks the ground truth by a different mechanism: it rasterises the wall solids at 0.05 m and flood-fills free space at door height, asking whether one room's interior physically reaches another's, rather than reasoning about polygon strips as the pipeline does. Agreement is 2,769/2,769 room pairs (100.0%).

2 Connectivity results

Method	Clean (L0/L1)	Damaged (L2)	Δ vs ours, clean
schema — read the schema as-is	0.0000	0.0000	+0.9956
adjacency — join every neighbour	0.6321	0.5600	+0.3635
door_near — nearest two rooms	0.9107	0.8272	+0.0849
Ifc2Graph (external, published)	0.5010	0.4760	+0.4946
ours	0.9956	0.8678	—

Table 1: Connectivity F1 against verified ground truth, 150 buildings. All differences vs. ours are significant at $p < 10^{-6}$ (paired Wilcoxon). Reproduce with main/significance.py –tag synth150.

Against door_near, the strongest in-repo baseline, the margin is 131 wins / 0 losses / 19 ties on clean input (95% CI $[+0.074, +0.096]$) and 82/2/66 under L2 damage ($[+0.032, +0.050]$).

schema scores 0.000 at every level, and that is the point. The schema-trusting baseline is what most BIM-to-graph work does. It recovers nothing because there is nothing in the file to read — the same fact the audit above reports, made operational.

The trivial baseline is not trivially beaten. adjacency achieves high recall by construction, so its precision is the number that matters: 0.486 on clean input, joining through 1,211 solid walls. This is the bar any real method must clear, and it is why the generator emits negatives.

3 Damage sweep

main/damage_ifc.py applies graduated damage, each level reproducing a pathology measured in the real corpus at its measured prevalence rather than an invented one: L1 strips space boundaries (100% of real models lack them — and a no-op on generated files, which never had any); L2 deletes IfcSpace on 49% of storeys; L3 orphans the survivors from their storey; L4 additionally drops all containment and rescales coordinates by 100 with the unit assignment detached.

L2 and L3 land identically across all 150 buildings (ours 0.8678). L4 differs on exactly two buildings, synth_0012 ($0.880 \to 0.917$) and synth_0135 ($0.963 \to 1.000$) — both marginally higher, mean 0.8683; the mechanism has not been investigated. The finding is that hierarchy repair and empirical unit detection absorb L3 and L4 essentially completely, so only missing rooms degrade connectivity. Room recovery, not hierarchy or unit repair, is the binding constraint — confirmed independently by this sweep, by the baseline ablation, and by the distribution of real-corpus failures.

4 External baselines

Ifc2Graph (scored). Bjørnskov's published, open-source tool voxelises IfcSpace meshes and reports two spaces adjacent if they share a voxel boundary. Run out of the box (one one-line NumPy 2.x compatibility patch) across all 300 buildings, zero crashes. It scores below our own naive adjacency ablation on clean input: precision 0.335 at its default 0.5 m voxel pitch — coarser than our 0.20 m walls — and it counts rooms merely stacked with a floor slab between them as adjacent (verified directly: it reports S0R000 -> S1R000 with no stair near either room). Its recall collapses $1.000 \to 0.436$ under L2 damage: having no room-recovery stage, IfcSpace stripping makes true connections literally invisible to it. Vertical-connection recall goes $150/150 \to 0/150$.

IFC2SB (built, run, not scorable). RWTH-E3D's C++/OpenCASCADE tool generates space boundaries from raw geometry. Building it required three fixes to the vendor's own files, none algorithmic: a Dockerfile FROM line with an inline comment that current BuildKit rejects; an ADD with no following WORKDIR; and src/Clip.cpp's dependency on CGAL, absent from the README's prerequisites. It then crashes identically — same exception, same token position — on both synthetic buildings tested, producing no output on any of the 300 ground-truth-verified buildings. On real files behaviour depends on how much IfcSpace the input already states: with 53 stated rooms it produces real interior data (51 spaces, 3,058 boundaries, 81 candidate connections) before a late segfault; with 2 stated rooms it completes cleanly but yields zero interior rooms.

There is no corpus on which both "IFC2SB runs" and "we have verified ground truth" hold simultaneously, so no F1 exists to report. It is excluded from the table above rather than misrepresented.

5 Real corpus: necessary conditions, not accuracy

The 25 real models yield 1,025 rooms (681 stated, 344 recovered) and 161 vertical links; every graph loads, and navigation and QnA function on all 25. Validation is by necessary conditions (main/validate.py) — conditions any correct building graph must satisfy:

	Condition	Pass
R2	every door joins exactly two rooms	25/25
R3	every room has one storey parent	25/25
R4	areas and storey heights are physical	25/25
R6	elements lie inside their assigned room	25/25
R1a	intra-storey connectivity ≥ 0.90	9/25
R1b	storeys mutually reachable	10/24 (1 n/a)
R5	element–room coverage ≥ 0.50	18/25

These cannot rank a graph, only falsify one. Nothing here establishes that a recovered corridor corresponds to a real corridor. The temptation is to read intra-storey reachability as a quality score; it is not one, and the corpus proves it — the adjacency baseline, which joins through 79 solid walls on known ground truth, scores 0.935 on that metric against our 0.744. A metric that rewards over-connection ranks the worst method first.

R1a failures are concentrated and explicable. They fall predominantly on open-plan, glazed buildings whose enclosure is curtain wall rather than IfcWall — one model holds 2,098 IfcPlate against 51 IfcWall — where there are few partitions to recover at all. A separate probe (main/proxy_probe.py) tested whether misclassified IfcBuildingElementProxy elements were the cause: a geometric signature recognising 98.1% of known IfcWall found the proxy population bimodal, and reclassifying all 259 wall-like proxies changed rooms, doors and reachability by exactly zero. Ruled out, not assumed away.

6 Goal resolution: the loop closed end to end

Every result above scores one stage. main/eval_goal_resolution.py asks the question the system exists to answer: given a functional query, is the right room retrieved, and can a specific robot then reach it? Using the same leave-one-out masking as the labelling study, resolution succeeds on $308/593 = 0.519$ of held-out real rooms, tracking per-class label accuracy closely because it is the same prediction asked differently. For each resolved goal, planning from the building entry succeeds for 0.169 of goals under a wheeled profile and 0.461 under legged/human.

The honest caveat: any candidate of the right class counts as resolved. This measures finding a room of the right kind, not disambiguating "the kitchen on the 2nd floor" from three others.

7 A partner delivery, verified and declined

Ramblr (partner company) ran their own IFC-to-scene-graph pipeline over 208 real models — of which our 25 are an exact byte-for-byte subset — and proposed the output as pseudo ground truth, to skip annotation.

We re-derived every number in their accuracy report from the raw files, using IfcRelVoidsElement + IfcRelFillsElement chained through the shared opening as an independent source of truth. Two counting bugs in our own first attempts (element nodes repeated once per assigned room; each relation stored on both endpoints) gave answers 6× and 2× off before being found. After fixing them, every aggregate matches exactly: 3,510 relations checked, 3,103 confirmed, precision 0.884, 333 extra-wall and 13 true mistakes and 61 unjudgeable, 22,671 real pairs, recall 0.137, 179,020 element nodes with zero broken references, and the full per-class element-coverage table down to the integer.

The report is accurate; the data is still not usable as connectivity ground truth, for two independent reasons. It asserts the wrong relation — wall contains opening, not which two rooms a door joins. And it is an open-world set: 86.3% of real wall-opening relations are absent and 92/208 files hold none at all, so treating absence as evidence of non-existence would penalise any method, ours included, for finding more correct connections than the reference happens to contain. The same trap as reading reachability as quality, in a different costume.

What verifying it did produce. The wall-opening relations we extracted to check Ramblr's work are themselves complete and native to the files: 22,671 real pairs across 208 models, no vendor tool required. This corpus also has substantial IfcSpace (188/208 files, median 71% of storeys) — materially richer than the 25-model subset's headline numbers suggest. Real, verifiable room-connectivity ground truth may therefore be derivable for these 208 buildings without manual annotation, by combining the native opening relations with room geometry and checking the result by the same independent-mechanism discipline used for the synthetic corpus. That is the largest open opportunity this project currently has.

8 What is still unmeasured

1. No accuracy figure on real files, for connectivity or for the 333 persisted room labels. Everything real is held-out or necessary-condition; everything accurate is synthetic.
2. Synthetic buildings are orthogonal and clean by design, to isolate connectivity recovery. Real files are harder, and the gap between 0.996 and what the method would score on annotated real data is unknown.
3. Room recovery on open-plan floors remains the binding constraint, and proxies have been ruled out as the cause without a replacement explanation being established.
4. Four label classes never resolve (outdoor, storage, retail, service), and goal resolution inherits that failure exactly.

9 Reproducing everything

Result	Command
Whole pipeline, end to end	./run_pipeline.sh
Schema audit (§1 numbers)	main/audit_dataset.py
Ground-truth verification	main/verify_gt.py –dir <corpus>
F1 table + significance	main/significance.py –tag synth150
External baseline scoring	main/baselines.py –methods ifc2graph
Real-corpus conditions	main/validate.py
Goal resolution	main/eval_goal_resolution.py
Ramblr re-verification	main/verify_ramblr_report.py

Companion reports: literature_review.pdf (positioning and the design reverse-engineering), semantic_injection_report.pdf (room labelling), benchmark_report.pdf (the zero-shot perception study that this design replaced).